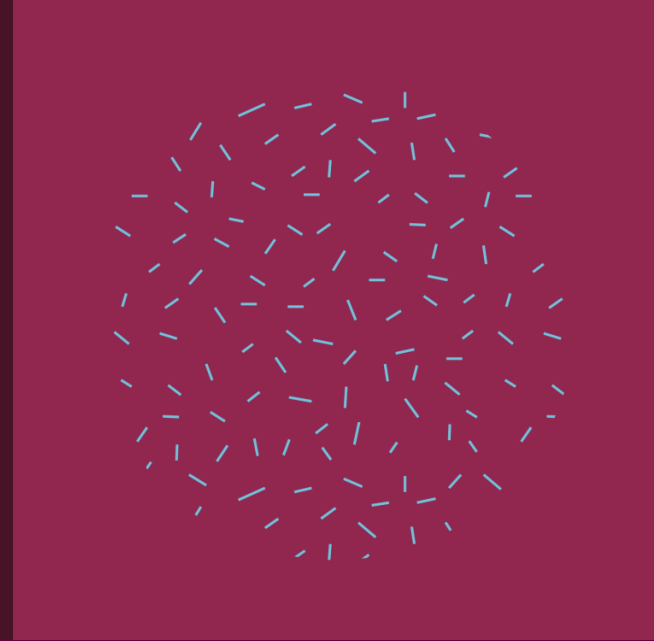




Data Wrangling

Data Collection , Integration and Statistics

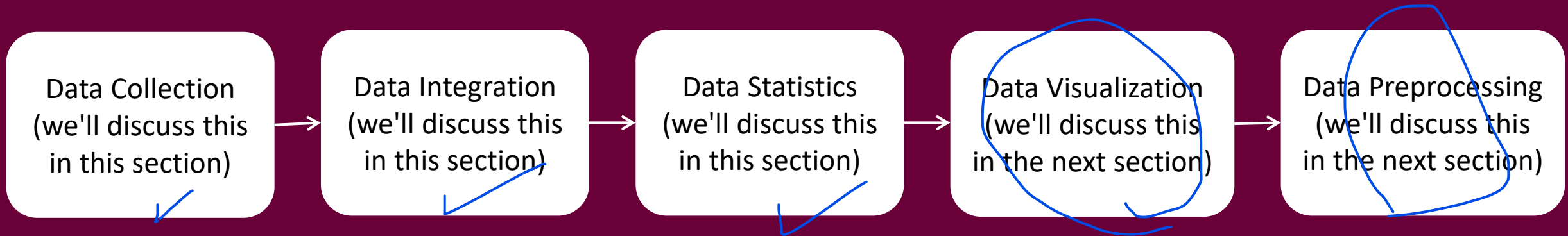
Section 1

Eng. Asmaa Fathy

What is Data Wrangling ?

- Data Wrangling is the process of **processing and transforming raw data into an orderly, clean form that can be** used to build accurate models or analytics.

- Tasks of Data Wrangling:



Data Collection

- Collecting Data From Files
- Collecting Data From The Web
- Collecting Data From SQL Databases



What is Data Collection ?

- Data Collection is Collecting data from different sources to be ready for analysis and modeling.
- Data Sources:
- Files: CSV, TXT, XLSX, JSON, XML, HTML
- Databases: MySQL, PostgreSQL, SQLite
- Web: Websites, Web Scraping



1. Collect Data from Files

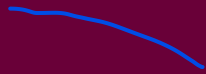
- We may have stored data in multiple types of files, such as `text`, `csv`, `excel`, `xml`, `html`, etc. We can load them into `dataframes`.

```
import pandas as pd
```



1.1 CSV

- You can get the path of your csv file, and feed the path to the function **read_csv**.



```
df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries.csv')
```



head(n)

- Displays the first n rows of data.
- If it is written as df.head() → by default n=5
- you can specify a different number: df.head(10) → the first 10 rows

df.head()

Unnamed: 0	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	company_name
0	0	2020	MI	FT	Data Scientist	70000	EUR	79833	DE	0	DE
1	1	2020	SE	FT	Machine Learning Scientist	260000	USD	260000	JP	0	JP
2	2	2020	SE	FT	Big Data Engineer	85000	GBP	109024	GB	50	GB
3	3	2020	MI	FT	Product Data Analyst	20000	USD	20000	HN	0	HN
4	4	2020	SE	FT	Machine Learning Engineer	150000	USD	150000	US	50	US

df.head(10)

Unnamed: 0	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	company_name
0	0	2020	MI	FT	Data Scientist	70000	EUR	79833	DE	0	DE
1	1	2020	SE	FT	Machine Learning Scientist	260000	USD	260000	JP	0	JP
2	2	2020	SE	FT	Big Data Engineer	85000	GBP	109024	GB	50	GB
3	3	2020	MI	FT	Product Data Analyst	20000	USD	20000	HN	0	HN
4	4	2020	SE	FT	Machine Learning Engineer	150000	USD	150000	US	50	US
5	5	2020	EN	FT	Data Analyst	72000	USD	72000	US	100	US
6	6	2020	SE	FT	Lead Data Scientist	190000	USD	190000	US	100	US
7	7	2020	MI	FT	Data Scientist	11000000	HUF	35735	HU	50	HU
8	8	2020	MI	FT	Business Data Analyst	135000	USD	135000	US	100	US
9	9	2020	SE	FT	Lead Data Engineer	125000	USD	125000	NZ	50	NZ

tail(n)

- Displays the last n rows of data.
- If it is written as df.tail() → by default n=5
- you can specify a different number: df.tail(10) → the last 10 rows

df.tail()

df.tail(10)

Unnamed: 0	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	
597	597	2022	SE	FT	Data Analyst	170000	USD	170000	US	100	US
598	598	2022	MI	FT	Data Scientist	160000	USD	160000	US	100	US
599	599	2022	MI	FT	Data Scientist	130000	USD	130000	US	100	US
600	600	2022	EN	FT	Data Analyst	67000	USD	67000	CA	0	CA
601	601	2022	EN	FT	Data Analyst	52000	USD	52000	CA	0	CA
602	602	2022	SE	FT	Data Engineer	154000	USD	154000	US	100	US
603	603	2022	SE	FT	Data Engineer	126000	USD	126000	US	100	US
604	604	2022	SE	FT	Data Analyst	129000	USD	129000	US	0	US
605	605	2022	SE	FT	Data Analyst	150000	USD	150000	US	100	US
606	606	2022	MI	FT	AI Scientist	200000	USD	200000	IN	100	US

Data Understanding

Column	Meaning	Type
work_year	The year the salary was paid.	Numeric
experience_level	The experience level in the job during the year with the following possible values: EN Entry-level / Junior MI Mid-level / Intermediate SE Senior-level / Expert EX Executive-level / Director	Categorical
employment_type	The type of employment for the role: PT Part-time FT Full-time CT Contract FL Freelance	Categorical
job_title	The role worked in during the year.	Categorical
salary	The total gross salary amount paid.	Numeric
salary_currency	The currency of the salary.	Categorical
salary_in_usd	The salary in USD.	Numeric
employee_residence	Employee's primary country of residence in during the work year.	Categorical
remote_ratio	The overall amount of work done remotely, possible values are as follows: 0 No remote work (less than 20%) 50 Partially remote 100 Fully remote (more than 80%)	Numeric
company_location	The country of the employer's main office or contracting branch.	Categorical
company_size	The average number of people that worked for the company during the year: S less than 50 employees (small) M 50 to 250 employees (medium) L more than 250 employees (large)	Categorical

df.info()

- A comprehensive summary of the DataFrame is displayed, including:
- RangeIndex (number of rows)
- Column names (Columns)
- Non-Null Count (number of non-null values in each column)
- Data type (Dtype) for each column
- Memory Usage

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 607 entries, 0 to 606
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	607 non-null	int64
1	work_year	607 non-null	int64
2	experience_level	607 non-null	object
3	employment_type	607 non-null	object
4	job_title	607 non-null	object
5	salary	607 non-null	int64
6	salary_currency	607 non-null	object
7	salary_in_usd	607 non-null	int64
8	employee_residence	607 non-null	object
9	remote_ratio	607 non-null	int64
10	company_location	607 non-null	object
11	company_size	607 non-null	object

```
dtypes: int64(5), object(7)
```

```
memory usage: 57.0+ KB
```

Numeric

Categorical

header=None

- Here we told pandas: “Don’t assume the first row contains column names.”
- Instead, pandas will treat all rows as data only and it will give the columns their default names: 0, 1, 2, ...
- When do we use header=None?
- If the file has no column names.
- If we want to manually change the column names after reading.

```
df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries.csv', header=None)
df.head()
```

	0	1	2	3	4	5	6	7	8	9	10	11
0	NaN	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	company_size
1	0.0	2020	MI	FT	Data Scientist	70000	EUR	79833	DE	0	DE	L
2	1.0	2020	SE	FT	Machine Learning Scientist	260000	USD	260000	JP	0	JP	S
3	2.0	2020	SE	FT	Big Data Engineer	85000	GBP	109024	GB	50	GB	M
4	3.0	2020	MI	FT	Product Data Analyst	20000	USD	20000	HN	0	HN	S

skiprows=1

- Skip the first row when reading
- This means pandas won't read the first row at all as data or a header.
- Useful if:
- The first row contains unimportant information (e.g., description or comment)
- You want to manually change the header

This dataset contains Data Science salaries	Table with 13 columns and 13 rows. Row 1: Unnamed: 1, Unnamed: 2, Unnamed: 3, Unnamed: 4, Unnamed: 5, Unnamed: 6, Unnamed: 7, Unnamed: 8, Unnamed: 9, Unnamed: 10, Unnamed: 11. Row 2: work_year, experience_level, employment_type, job_title, salary, salary_currency, salary_in_usd, employee_residence, remote_ratio, company_location, company_size.												
	0	NaN	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	company_size
	1	0.0	2020	MI	FT	Data Scientist	70000	EUR	79833	DE	0	DE	1
	2	1.0	2020	SE	FT	Machine Learning Scientist	260000	USD	260000	JP	0	JP	2
	3	2.0	2020	SE	FT	Big Data Engineer	85000	GBP	109024	GB	50	GB	3
	4	3.0	2020	MI	FT	Product Data Analyst	20000	USD	20000	HN	0	HN	4
	5	4.0	2020	SE	FT	Machine Learning Scientist	260000	USD	260000	JP	0	JP	5
	6	5.0	2020	SE	FT	Big Data Engineer	85000	GBP	109024	GB	50	GB	6
	7	6.0	2020	MI	FT	Product Data Analyst	20000	USD	20000	HN	0	HN	7
	8	7.0	2020	MI	FT	Data Scientist	70000	EUR	79833	DE	0	DE	8
	9	8.0	2020	SE	FT	Machine Learning Scientist	260000	USD	260000	JP	0	JP	9
	10	9.0	2020	SE	FT	Big Data Engineer	85000	GBP	109024	GB	50	GB	10
	11	10.0	2020	MI	FT	Product Data Analyst	20000	USD	20000	HN	0	HN	11

unimportant info

```
df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries2.csv', skiprows=1)
df.head()
```

	Unnamed: 0	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	company_size
0	0	2020	MI	FT	Data Scientist	70000	EUR	79833	DE	0	DE	1
1	1	2020	SE	FT	Machine Learning Scientist	260000	USD	260000	JP	0	JP	2
2	2	2020	SE	FT	Big Data Engineer	85000	GBP	109024	GB	50	GB	3
3	3	2020	MI	FT	Product Data Analyst	20000	USD	20000	HN	0	HN	4
4	4	2020	SE	FT	Machine Learning Engineer	150000	USD	150000	US	50	US	5

skipfooter=3

- Skip the last 3 rows when reading
- This means that Pandas will not read any of the last 3 rows in the file.
- This is very important for files that contain summaries, comments, or unimportant data at the end.
- You need to specify engine='python' because the default pandas C does not support skipfooter.

```
df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries2.csv', skiprows=1)
# df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries2.csv', skiprows=1, skipfooter=3, engine='python')
df.tail(5)
```

	Unnamed: 0	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_
605	605	2022	SE	FT	Data Analyst	150000.0	USD	150000.0	US	100.0	
606	606	2022	MI	FT	AI Scientist	200000.0	USD	200000.0	IN	100.0	
607	experience_level	The experience level in the job during the yea...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
608	employment_type	The type of employment for the role: PT Part...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
609	company_size	The average number of people that worked for t...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

unimportant info

```
# df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries2.csv', skiprows=1)
df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries2.csv', skiprows=1, skipfooter=3, engine='python')
df.tail(5)
```

	Unnamed: 0	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	c
602	602	2022	SE	FT	Data Engineer	154000	USD	154000	US	100	US	
603	603	2022	SE	FT	Data Engineer	126000	USD	126000	US	100	US	
604	604	2022	SE	FT	Data Analyst	129000	USD	129000	US	0	US	
605	605	2022	SE	FT	Data Analyst	150000	USD	150000	US	100	US	
606	606	2022	MI	FT	AI Scientist	200000	USD	200000	IN	100	US	

1.2 TXT

- If the txt follows csv format, then it can be read as a csv file

```
df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries.txt')  
df
```

1.3 Excel

- You can get the path of your excel file, and feed the path to the function **read_excel**.

```
df = pd.read_excel('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries.xlsx')  
df
```

1.4 Json

- You can get the path of your json file, and feed the path to the function **read_json**.

```
df = pd.read_json('C:/Users/Admin/Documents/Data Mining Practical/ds_salaries.json')  
df.head()
```


2.1 Web Scrapping

It is that we retrieve data from a website using code instead of copying it manually.

- We'll need to first import the **requests** library, and then download the page using the **requests.get** method:

```
import requests
page = requests.get("https://dataquestio.github.io/web-scraping-pages/simple.html")
page

<Response [200]>
```

- After running our request, we get a Response object. This object has a **status_code** property, which indicates if the page was downloaded successfully:

```
page.status_code
200
```

A **status_code** of 200 means that the **page downloaded successfully**. A status code starting with a 2 generally indicates success, and a code starting with a 4 or a 5 indicates an error.

- We can print out the HTML content of the page using the **content** property:

```
page.content

b'<!DOCTYPE html>\n<html>\n  <head>\n    <title>A simple example page</title>\n  </head>\n  <body>\n    <p>Here is some simple content for this page.</p>\n  </body>\n</html>'
```

2.2 BeautifulSoup library

- Parsing by **BeautifulSoup** As you can see above, we now have downloaded an HTML document.
- We can use the BeautifulSoup library to **parse** this document and extract **the text from the p tag**.

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(page.content, 'html.parser')
```

- We can now print out the HTML content of the page, formatted nicely, using the **prettify** method on the BeautifulSoup object.

```
print(soup.prettify())

<!DOCTYPE html>
<html>
  <head>
    <title>
      A simple example page
    </title>
  </head>
  <body>
    <p>
      Here is some simple content for this page.
    </p>
  </body>
</html>
```

2.3 Finding Tags

- Finding all instances of a tag at once What we did above was useful for figuring out how to navigate a page, but it took a lot of commands to do something fairly simple.
- If we want to extract a single tag, we can instead use the `find_all` method, which will find all the instances of a tag on a page.

if we are looking for the title, we can look for <title> tag

```
soup.find_all('title')
```

```
[<title>A simple example page</title>]
```

```
for t in soup.find_all('title'):
    print(t.get_text())
```

```
A simple example page
```

If we are looking for text, we can look for <p> tag

```
for t in soup.find_all('p'):
    print(t.get_text())
```

```
Here is some simple content for this page.
```

If you instead only want to find the first instance of a tag, you can use the `find` method, which will return a single BeautifulSoup object:

```
soup.find('p').get_text()
```

```
'Here is some simple content for this page.'
```

2.4 Searching for tags by class and id

Classes and ids are used by CSS to determine which HTML elements to apply certain styles to. But when we're scraping, we can also use them to specify the elements we want to scrape.

```
page = requests.get("https://dataquestio.github.io/web-scraping-pages/ids_and_classes.html")
soup = BeautifulSoup(page.content, 'html.parser')
soup
```

```
<html>
<head>
<title>A simple example page</title>
</head>
<body>
<div>
<p class="inner-text first-item" id="first">
    First paragraph.
</p>
<p class="inner-text">
    Second paragraph.
</p>
</div>
<p class="outer-text first-item" id="second">
<b>
    First outer paragraph.
</b>
</p>
<p class="outer-text">
<b>
    Second outer paragraph.
</b>
</p>
</body>
</html>
```

Now, we can use the `find_all` method to search for items by class or by id. In the below example, we'll search for any `p` tag that has the class `outer-text`:

```
soup.find_all('p', class_='outer-text')

[<p class="outer-text first-item" id="second">
  <b>
    First outer paragraph.
  </b>
</p>,
<p class="outer-text">
  <b>
    Second outer paragraph.
  </b>
</p>]
```

In the below example, we'll look for any tag that has the class `outer-text`:

```
soup.find_all(class_="outer-text")

[<p class="outer-text first-item" id="second">
  <b>
    First outer paragraph.
  </b>
</p>,
<p class="outer-text">
  <b>
    Second outer paragraph.
  </b>
</p>]
```

We can also search for elements by id:

```
soup.find_all(id="first")

[<p class="inner-text first-item" id="first">
    First paragraph.
</p>]
```

3. Collect Data from Databases (SQLite)

- A file with the .sqlite extension is a lightweight SQL database file created with the SQLite software.
- It is a database in a file itself and implements a self-contained, full-featured, highly-reliable SQL database engine.
- We use SQLite to demonstrate the approach to access SQL databases. They follow similar steps. You just need to setup your account credentials in the connect so you can connect the server.



3.1 Read an SQLite Database in Python

- We use a Python package, `sqlite3`, to deal with SQLite databases. Once the `sqlite3` package is imported, the general steps are:
1. Create a connection object that connects the SQLite database.
 2. Create a cursor object
 3. Create a query statement
 4. Execute the query statement
 5. Fetch the query result to result
 6. If all work is done, close the connection.

```
df = pd.read_csv("C:/Users/Admin/Documents/Data Mining Prac

# Create Database
connection = sqlite3.connect("ds_salaries.sqlite")

# Save CSV Data as SQL
df.to_sql("salaries", connection, index=False)

connection.close()
```

```
import sqlite3
connection = sqlite3.connect('C:/Users/Admin/Documents/Data
cursor = connection.cursor()

query = '''
SELECT name FROM sqlite_master
WHERE type='table';
'''

cursor.execute(query)
results = cursor.fetchall()
results

[('salaries',)]
```

3.2 Play with the SQLite Databases

- Using SQL statements, you can play with the SQLite Databases and get the data you need.

```
[63]: query = '''SELECT *  
FROM salaries'''  
cursor.execute(query)  
results = cursor.fetchall()  
results
```

```
[63]: [(0,  
       2020,  
       'MI',  
       'FT',  
       'Data Scientist',  
       70000,  
       'EUR',  
       79833,  
       'DE',  
       0,  
       'DE',  
       'L'),  
      (1,  
       2020,  
       'SE',  
       'FT',  
       'Machine Learning Scientist',  
       260000
```


3.3 Save Data to CSV Files

```
df = pd.DataFrame(results)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 607 entries, 0 to 606
Data columns (total 12 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0    0      607 non-null    int64
 1    1      607 non-null    int64
 2    2      607 non-null    object
 3    3      607 non-null    object
 4    4      607 non-null    object
 5    5      607 non-null    int64
 6    6      607 non-null    object
 7    7      607 non-null    int64
 8    8      607 non-null    object
 9    9      607 non-null    int64
10   10     607 non-null    object
11   11     607 non-null    object
dtypes: int64(5), object(7)
memory usage: 57.0+ KB
```

To close the connection:

```
cursor.close()
connection.close()
```

Data Integration

- Data Integration is the process of combining data from different sources into a single, unified view.
- It involves the combination of data from different data types, structures, and formats to form a single dataset that can be used for analysis and reporting.
- This step is important because it allows for the analysis of data from multiple sources, which can provide a complete and more accurate picture of the data being analyzed.



```
import numpy as np
import pandas as pd

df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/housing.csv')
df.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   longitude             20640 non-null  float64
1   latitude              20640 non-null  float64
2   housing_median_age    20640 non-null  float64
3   total_rooms           20640 non-null  float64
4   total_bedrooms        20433 non-null  float64
5   population             20640 non-null  float64
6   households             20640 non-null  float64
7   median_income         20640 non-null  float64
8   median_house_value    20640 non-null  float64
9   ocean_proximity       20640 non-null  object  
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

- 1. longitude:** A measure of how far west a house is; a higher value is farther west
- 2. latitude:** A measure of how far north a house is; a higher value is farther north
- 3. housingMedianAge:** Median age of a house within a block; a lower number is a newer building
- 4. totalRooms:** Total number of rooms within a block
- 5. totalBedrooms:** Total number of bedrooms within a block
- 6. population:** Total number of people residing within a block
- 7. households:** Total number of households, a group of people residing within a home unit, for a block
- 8. medianIncome:** Median income for households within a block of houses (measured in tens of thousands of US Dollars)
- 9. medianHouseValue:** Median house value for households within a block (measured in US Dollars)
- 10. oceanProximity:** Location of the house w.r.t ocean/sea

1. Concatenation

```
# made a subset of columns and select 5 random rows of data.
df_1=df[['longitude','latitude','median_income']].sample(n=5)
df_2=df[['longitude','latitude','median_income']].sample(n=5)
df_3=df[['longitude','latitude','median_income']].sample(n=5)
```

df_1			
	longitude	latitude	median_income
2071	-119.85	36.74	3.5375
18089	-122.06	37.33	7.0102
15124	-116.88	32.86	5.0733
16541	-121.23	37.79	4.0815
17577	-121.94	37.31	4.0474

df_2			
	longitude	latitude	median_income
11070	-117.85	33.84	5.1785
12716	-121.37	38.60	3.7880
9165	-118.43	34.42	6.2971
292	-122.16	37.77	3.5739
11298	-117.91	33.78	4.0362

df_3			
	longitude	latitude	median_income
2568	-124.15	40.79	3.0216
20044	-119.03	36.06	2.1620
3083	-118.44	35.75	2.6288
1476	-122.04	37.96	2.9097
7089	-118.00	33.94	3.7500

- Concatenation** is used to combine multiple DataFrames either **vertically** (adding rows) or **horizontally** (adding columns).

```
# Vertical Concatenation
df_cat1=pd.concat([df_1,df_2,df_3],axis=0)
df_cat1
```

	longitude	latitude	median_income
2071	-119.85	36.74	3.5375
18089	-122.06	37.33	7.0102
15124	-116.88	32.86	5.0733
16541	-121.23	37.79	4.0815
17577	-121.94	37.31	4.0474
11070	-117.85	33.84	5.1785
12716	-121.37	38.60	3.7880
9165	-118.43	34.42	6.2971
292	-122.16	37.77	3.5739
11298	-117.91	33.78	4.0362
2568	-124.15	40.79	3.0216
20044	-119.03	36.06	2.1620
3083	-118.44	35.75	2.6288
1476	-122.04	37.96	2.9097
7089	-118.00	33.94	3.7500

```
df_cat1.info()

<class 'pandas.core.frame.DataFrame'>
Index: 15 entries, 2071 to 7089
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0    longitude    15 non-null     float64
1    latitude     15 non-null     float64
2    median_income 15 non-null     float64
dtypes: float64(3)
memory usage: 480.0 bytes
```

```
# Horizontal Concatenation
df_cat2=pd.concat([df_1,df_2,df_3],axis=1)
df_cat2
```

	longitude	latitude	median_income	longitude	latitude	median_income	longitude	latitude	median_income
2071	-119.85	36.74	3.5375	NaN	NaN	NaN	NaN	NaN	NaN
18089	-122.06	37.33	7.0102	NaN	NaN	NaN	NaN	NaN	NaN
15124	-116.88	32.86	5.0733	NaN	NaN	NaN	NaN	NaN	NaN
16541	-121.23	37.79	4.0815	NaN	NaN	NaN	NaN	NaN	NaN
17577	-121.94	37.31	4.0474	NaN	NaN	NaN	NaN	NaN	NaN
11070	NaN	NaN	NaN	-117.85	33.84	5.1785	NaN	NaN	NaN
12716	NaN	NaN	NaN	-121.37	38.60	3.7880	NaN	NaN	NaN
9165	NaN	NaN	NaN	-118.43	34.42	6.2971	NaN	NaN	NaN
292	NaN	NaN	NaN	-122.16	37.77	3.5739	NaN	NaN	NaN
11298	NaN	NaN	NaN	-117.91	33.78	4.0362	NaN	NaN	NaN
2568	NaN	NaN	NaN	NaN	NaN	NaN	-124.15	40.79	3.0216
20044	NaN	NaN	NaN	NaN	NaN	NaN	-119.03	36.06	2.1620
3083	NaN	NaN	NaN	NaN	NaN	NaN	-118.44	35.75	2.6288
1476	NaN	NaN	NaN	NaN	NaN	NaN	-122.04	37.96	2.9097
7089	NaN	NaN	NaN	NaN	NaN	NaN	-118.00	33.94	3.7500

```
df_cat2.info()

<class 'pandas.core.frame.DataFrame'>
Index: 15 entries, 2071 to 7089
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0    longitude    5 non-null     float64
1    latitude     5 non-null     float64
2    median_income 5 non-null     float64
3    longitude    5 non-null     float64
4    latitude     5 non-null     float64
5    median_income 5 non-null     float64
6    longitude    5 non-null     float64
7    latitude     5 non-null     float64
8    median_income 5 non-null     float64
dtypes: float64(9)
memory usage: 1.2 KB
```

Since `.sample()` selected random rows, the indexes in `df_1`, `df_2`, and `df_3` are likely not identical. pandas performs alignment based on the index. If the index is different: NaN values will be displayed.

- We selected the first 5 rows. Each DataFrame has only one column. They all have the same index.

```
df_1 = df[['longitude']][:5]  
df_2 = df[['latitude']][:5]  
df_3 = df[['median_income']][:5]
```

```
df_1, df_2, df_3
```

```
( longitude  
0    -122.23  
1    -122.22  
2    -122.24  
3    -122.25  
4    -122.25,  
 latitude  
0     37.88  
1     37.86  
2     37.85  
3     37.85  
4     37.85,  
 median_income  
0      8.3252  
1      8.3014  
2      7.2574  
3      5.6431  
4      3.8462)
```

```
# Horizontal Concatenation (Number of rows = 5 and the index is identical)  
df_cat2 = pd.concat([df_1, df_2, df_3], axis=1)  
df_cat2
```

	longitude	latitude	median_income
0	-122.23	37.88	8.3252
1	-122.22	37.86	8.3014
2	-122.24	37.85	7.2574
3	-122.25	37.85	5.6431
4	-122.25	37.85	3.8462

2. Merging

- **Merging** combines DataFrames based on a **common key**, similar to SQL JOIN operations.

```
df_1=df[['longitude','median_income']][0:5]
```

```
df_1
```

	longitude	median_income
0	-122.23	8.3252
1	-122.22	8.3014
2	-122.24	7.2574
3	-122.25	5.6431
4	-122.25	3.8462

```
df_2=df[['longitude','median_house_value']][0:5]
```

```
df_2
```

	longitude	median_house_value
0	-122.23	452600.0
1	-122.22	358500.0
2	-122.24	352100.0
3	-122.25	341300.0
4	-122.25	342200.0

```
pd.merge(df_1, df_2, on=['longitude'], how='inner')
```

	longitude	median_income	median_house_value
0	-122.23	8.3252	452600.0
1	-122.22	8.3014	358500.0
2	-122.24	7.2574	352100.0
3	-122.25	5.6431	341300.0
4	-122.25	5.6431	342200.0
5	-122.25	3.8462	341300.0
6	-122.25	3.8462	342200.0

inner	shared only
left	all rows of Table 1
right	all rows of Table 2
outer	all rows of both

3. Joining

- **Joining** aligns two DataFrames on their **index**.

```
df_1=df[['longitude']][0:5]  
df_1
```

	longitude
0	-122.23
1	-122.22
2	-122.24
3	-122.25
4	-122.25

```
df_2=df[['latitude']][2:7]  
df_2
```

	latitude
2	37.85
3	37.85
4	37.85
5	37.85
6	37.84

```
df_1.join(df_2,how='left')
```

	longitude	latitude
0	-122.23	NaN
1	-122.22	NaN
2	-122.24	37.85
3	-122.25	37.85
4	-122.25	37.85

```
df_1.join(df_2,how='right')
```

	longitude	latitude
2	-122.24	37.85
3	-122.25	37.85
4	-122.25	37.85
5	NaN	37.85
6	NaN	37.84

```
df_1.join(df_2,how='inner')
```

	longitude	latitude
2	-122.24	37.85
3	-122.25	37.85
4	-122.25	37.85

```
df_1.join(df_2,how='outer')
```

	longitude	latitude
0	-122.23	NaN
1	-122.22	NaN
2	-122.24	37.85
3	-122.25	37.85
4	-122.25	37.85
5	NaN	37.85
6	NaN	37.84

Data Statistics

- Data Statistical Understanding is the process of gaining insights and knowledge about the data through statistical analysis.
- This step involves tasks such as descriptive statistics, probability distributions, hypothesis testing, and inferential statistics.
- The goal is to understand the characteristics of the data, identify patterns, and make predictions about future data.



1. DESCRIPTIVE DATA ANALYSIS

- it's a fundamental step in Data Understanding before we begin any modeling.
- Data Types :We divide data into two main types:

Type	Example	Description
Numerical	median_income, population	Quantitative numbers that can be used for calculations (sum, average, etc.)
Non-numerical / Categorical	oceanProximity	Qualitative data, such as categories or labels (e.g., NEAR OCEAN, INLAND)

- Knowing the data type is very important to choose the right method for analysis and processing.

1.1 Central Tendency

- A number that represents the entire data level and give us an idea of the central point of the data.
- Key Measures: Mean, Median, and Mode.

Measure	Meaning	Example
✓ Mean	The average of values. The sum of the values ÷ the number of values	Imagine we have 5 students with the following grades: 50, 60, 70, 80, 90 $(50+60+70+80+90) \div 5 = 70$ The "overall level" for the class is approximately 70.
✓ Median	The value in the middle after sorting the data	Imagine we have 5 students with the following grades: 50, 60, 70, 80, 90 We arrange the grades (they are already arranged) Median = 70 → This is the grade in the middle.
✓ Mode	The most frequently occurring value	Imagine we have 5 students with the following grades: 50, 60, 70, 70, 90 70 appeared twice → is the Mode

What if there's an outlier? Example: 50, 60, 70, 80, 200

Value = 200 → Outlier

The mean will be: $(50+60+70+80+200) \div 5 = 92$

The mean is now 92 even though most of the scores are lower!

But the median is still 70. That's why the median is stronger when there are outliers.

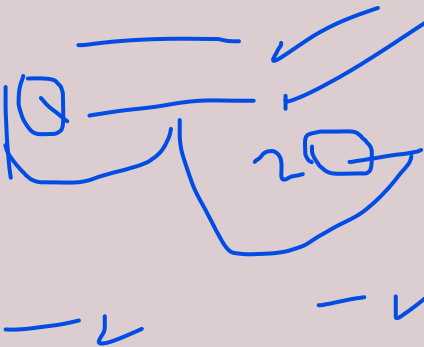
1.2 Dispersion

- Measures how dispersed or closely spaced the data is.
- In other words, Is there a significant difference between the values, or not?
- Key Measures: Variance , Standard Deviation. They are the same idea but with two different methods of calculation.

Measure	Meaning	Example
Variance	The average of the squared differences from the mean	Values: 10 , 12 , 14 1- We calculate the mean: $(10 + 12 + 14) \div 3 = 12$ 2- We calculate the distance from the mean: $10 - 12 = -2$ $12 - 12 = 0$ $14 - 12 = +2$ 3- We square the differences: $(-2)^2 = 4$ $0^2 = 0$ $2^2 = 4$ 4- Their mean: $(4 + 0 + 4) \div 3 = 2.67 \rightarrow$ Variance
Standard Deviation	The Root of the Variance	Taking (square root of 2.67) $\approx 1.63 \rightarrow$ Standard Deviation

1.3 Location

- The data is divided into parts (25% – 50% – 75%)
- Helps us identify important centers in the data, such as Quartiles (25%, 50%, 75%)
- **Example Imagine we have 8 ranked degrees: 10, 20, 30, 40, 50, 60, 70, 80**
- Q1 First Quartile (25%): One-quarter of the data $\rightarrow (20+30)/2 = 25$
- Q2 (50%): Same as the Median $\rightarrow 45$
- Q3 Third Quartile (75%): Three-quarters of the data $\rightarrow (60+70)/2 = 65$

Measure	Meaning	Example
Interquartile Range (IQR) IQR = Q3 - Q1 	The distance between the first quarter (Q1) and the third quarter (Q3) This measures uses to detect outliers. Why is it called the middle half? Because: The first 25% of values before Q1 , The last 25% after Q3 and the 50% in between is the important part This means we ignore: The lowest quartile and The highest quartile (which might contain outliers)	Imagine we have this data: 5, 7, 8, 9, 10, 11, 12, 100 100 is a slightly unusual number (outlier) When we calculate: Q1 ≈ 7.5 Q3 ≈ 11.5 So: IQR = $11.5 - 7.5 = 4$ This means that most natural data lies between 7.5 and 11.5 $\rightarrow$ This is the "middle half".

Load Data

```
df = pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/Spotify_Youtube_Sample.csv')
df.head()
```

	Artist	Track	Album	Album_type	Views	Likes	Comments	Licensed	official_video	Stream
0	Ed Sheeran	Track 0	Album 0	album	341677198.0	5292328.0	252495.0	1.0	1.0	674649029.0
1	Billie Eilish	Track 1	Album 1	single	64929802.0	7200067.0	485193.0	0.0	NaN	742067481.0
2	Adele	Track 2	Album 2	EP	275211724.0	1923054.0	94003.0	1.0	0.0	713031508.0
3	Billie Eilish	Track 3	Album 3	album	694174172.0	9146026.0	363189.0	1.0	NaN	516467840.0
4	Billie Eilish	Track 4	Album 4	EP	137636237.0	4005416.0	471348.0	NaN	1.0	78519502.0

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20718 entries, 0 to 20717
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Artist          20718 non-null object
 1   Track           20718 non-null object
 2   Album           20718 non-null object
 3   Album_type      20718 non-null object
 4   Views           20248 non-null float64
 5   Likes           20177 non-null float64
 6   Comments        20149 non-null float64
 7   Licensed        20316 non-null float64
 8   official_video  20263 non-null float64
 9   Stream          20142 non-null float64
dtypes: float64(6), object(4)
memory usage: 1.6+ MB
```

df.describe()

- It gives you a quick statistical summary of the numerical column in the data.
- Instead of looking at all 20,718 rows, it summarizes the important values like the mean, minimum, maximum, etc.

df.describe()						
	Views	Likes	Comments	Licensed	official_video	Stream
count	2.024800e+04	2.017700e+04	20149.000000	20316.000000	20263.000000	2.014200e+04
mean	4.971178e+08	4.972103e+06	251616.443347	0.502560	0.494399	5.028411e+08
std	2.891963e+08	2.875093e+06	143712.257532	0.500006	0.499981	2.880925e+08
min	2.480200e+04	3.330000e+02	23.000000	0.000000	0.000000	9.464000e+03
25%	2.458302e+08	2.475846e+06	129287.000000	0.000000	0.000000	2.545154e+08
50%	4.953072e+08	4.947856e+06	251992.000000	1.000000	0.000000	5.030557e+08
75%	7.462919e+08	7.430760e+06	375083.000000	1.000000	1.000000	7.528422e+08
max	9.999693e+08	9.999633e+06	499960.000000	1.000000	1.000000	9.999828e+08

Number of non-null values in the column

Arithmetic mean (sum ÷ number of values)

shows how far the data deviates from the mean

Smallest value in the column

First quartile (Q1) – 25% of values are below this value

Median – Half of the values are below this value and half are above it

Third quartile (Q3) – 75% of the values are below this value

Largest value in the column

A. Non-Numerical Attributes (text/boolean)

- **value_counts()** → How many times does each element appear?
- This gives you the number of times each artist appears in the `Artist` column.
- Useful for identifying the most represented artists in your data.

```
df['Artist'].value_counts()
```

```
Artist
Drake      4229
Gorillaz   4182
Billie Eilish 4174
Ed Sheeran 4126
Adele      4007
Name: count, dtype: int64
```

- **value_counts()** , **unique()** , **nunique()** :
Using them with non-numerical columns
→ helps you understand data diversity before analysis.

- **unique()** → What are the different elements?
- This gives you all the different (unique) values in the `Artist` column.

```
df['Artist'].unique()
```

```
array(['Ed Sheeran', 'Billie Eilish', 'Adele', 'Drake', 'Gorillaz'],  
      dtype=object)
```

- **nunique()** → How many different elements are there?
- This gives you the number of unique values in the column.
- This is faster when you don't need to see the names themselves but just want to know the number of different artists.

```
df['Artist'].nunique()
```

```
5
```

```
nonnumericalcols = ['Artist', 'Track', 'Album', 'Album_type', 'Licensed', 'official_video']  
df[nonnumericalcols].nunique()
```

```
Artist      5  
Track      20718  
Album      100  
Album_type   3  
Licensed     2  
official_video 2  
dtype: int64
```

B. Categorical Attributes

- Columns with specific and repeated values such as:
 - ✓ Album_type → album, single, compilation
 - ✓ Licensed → True / False
 - ✓ official_video → True / False
- These are different from numbers because we don't calculate sums or averages them; we see their distribution.

value_counts(): this shows you how many times each category appears in the column.

```
album_type = pd.DataFrame({'Album_type' : df['Album_type'].value_counts()})  
album_type
```

Album_type	
Album_type	
album	6958
EP	6957
single	6803

```
licensed = pd.DataFrame({'Licensed' : df['Licensed'].value_counts()})  
licensed
```

Licensed	
Licensed	
1.0	10210
0.0	10106

```
official_video = pd.DataFrame({'official_video' : df['official_video'].value_counts()})  
official_video
```

official_video	
official_video	
0.0	10245
1.0	10018

Numerical Attributes : Central Tendency

min, max, median, mode, **midrange** → **$(\text{max} - \text{min})/2$**

```
col = 'Views'
min = df[col].min()
max = df[col].max()
median = df[col].median()
mode = df[col].mode()[0]
midrange = (max - min)/2
print('col:', col,
      '\n\tmin:', min,
      'max:', max,
      'median:', median,
      'mode:', mode,
      'midrange:', midrange)
```

```
col: Views
      min: 24802.0 max: 999969278.0 median: 495307151.0 mode: 648254656.0 midrange: 499972238.0
```

C. Numerical Attributes

```
def getCentralTendency(col):
    min = df[col].min()
    max = df[col].max()
    median = df[col].median()
    mode = df[col].mode()[0]
    midrange = (max - min)/2
    print('col:', col,
          '\n\tmin:', min,
          'max:', max,
          'median:', median,
          'mode:', mode,
          'midrange:', midrange)
```

```
numericalcols = ['Views', 'Likes', 'Comments', 'Stream']
```

```
for col in numericalcols:
    getCentralTendency(col)
```

```
col: Views
      min: 24802.0 max: 999969278.0 median: 495307151.0 mode: 648254656.0 midrange: 499972238.0
col: Likes
      min: 333.0 max: 9999633.0 median: 4947856.0 mode: 171528.0 midrange: 4999650.0
col: Comments
      min: 23.0 max: 499960.0 median: 251992.0 mode: 64376.0 midrange: 249968.5
col: Stream
      min: 9464.0 max: 999982809.0 median: 503055741.5 mode: 9464.0 midrange: 499986672.5
```

The difference between max and min → shows the range of values

Numerical Attributes : Dispersion

range, quantiles, var, std

```
col = 'Views'
range = df[col].max()- df[col].min()
quantiles = df[col].quantile([0.25, 0.5, 0.75])
IQR = quantiles[0.75]- quantiles[0.25]
var = df[col].var()
std = df[col].std()
print('col:',col,
      '\n\trange:', range,
      'Q1:',quantiles[0.25],
      'Q2:', quantiles[0.5],
      'Q3:', quantiles[0.75],
      'IQR:', IQR,
      'var:', var,
      'std:', std)
```

```
col: Views
      range: 999944476.0 Q1: 245830150.0 Q2: 495307151.0 Q3: 746291899.5 IQR: 500461749.5 var: 8.36345048970017e+16 std: 289196308.5812156
```

```
def getDispersion(col):
    range = df[col].max()- df[col].min()
    quantiles = df[col].quantile([0.25, 0.5, 0.75])
    IQR = quantiles[0.75]- quantiles[0.25]
    var = df[col].var()
    std = df[col].std()
    print('col:',col,
          '\n\trange:', range,
          'Q1:',quantiles[0.25],
          'Q2:', quantiles[0.5],
          'Q3:', quantiles[0.75],
          'IQR:', IQR,
          'var:', var,
          'std:', std)
```

```
numericalcols = ['Views', 'Likes', 'Comments', 'Stream']
```

```
for col in numericalcols:
    getDispersion(col)
```

```
col: Views
      range: 999944476.0 Q1: 245830150.0 Q2: 495307151.0 Q3: 746291899.5 IQR: 500461749.5 var: 8.36345048970017e+16 std: 289196308.5812156
col: Likes
      range: 9999300.0 Q1: 2475846.0 Q2: 4947856.0 Q3: 7430760.0 IQR: 4954914.0 var: 8266160785640.884 std: 2875093.178601501
col: Comments
      range: 499937.0 Q1: 129287.0 Q2: 251992.0 Q3: 375083.0 IQR: 245796.0 var: 20653212964.971184 std: 143712.25753209495
col: Stream
      range: 999973345.0 Q1: 254515398.75 Q2: 503055741.5 Q3: 752842183.5 IQR: 498326784.75 var: 8.299727068638902e+16 std: 288092468.9858953
```

C. Numerical Attributes

C. Numerical Attributes

Numerical Attributes : Correlation

- `df[numericalcols].corr()` → Calculates the degree of correlation between numerical columns
- Result between -1 and 1:
- 1 → Strong and direct correlation
- -1 → Strong and inverse correlation
- 0 → Almost no correlation

```
df[numericalcols].corr()
```

	Views	Likes	Comments	Stream
Views	1.000000	-0.012206	0.000916	0.008073
Likes	-0.012206	1.000000	0.011111	-0.003257
Comments	0.000916	0.011111	1.000000	-0.000292
Stream	0.008073	-0.003257	-0.000292	1.000000

Summary

What is Data Wrangling?	Transform raw data into clean, usable form. Steps: Collection → Integration → Statistics → Visualization → Preprocessing
Data Collection	Files: CSV, TXT, Excel, JSON, XML, HTML → <code>pd.read_csv()</code> , <code>pd.read_excel()</code> , <code>pd.read_json()</code> Web: Requests + BeautifulSoup → extract data from websites Databases: SQLite, MySQL, PostgreSQL → <code>sqlite3.connect()</code> + SQL queries
Data Understanding	<code>df.head()</code>, <code>df.tail()</code> → view first/last rows <code>df.info()</code> → summary of columns, data types, null counts <code>df.describe()</code> → quick statistical summary of the numerical column in the data Know column types: Numerical: can calculate sum, mean, std, etc. Categorical / Non-numerical: categories or labels
Data Integration	Combine multiple sources into one dataset Techniques: Concatenation: stack DataFrames vertically (rows) or horizontally (columns) Merge: combine on a key (like SQL JOIN) → inner, outer, left, right Join: align DataFrames on index
Data Statistics (Descriptive Analysis)	Central Tendency: single number that represents the “middle”. Mean, Median, Mode, Midrange Dispersion: spread of data. Range, Variance, Standard Deviation, IQR Location / Quartiles: split data into Q1 (25%), Q2/Median (50%), Q3 (75%) Categorical Analysis: <code>value_counts()</code> , <code>unique()</code> , <code>nunique()</code> Correlation: <code>df.corr()</code> → relationship between numerical columns

Task

- Each student downloads a different dataset (CSV, Excel, JSON... anything suitable) and tries to understand it and apply the steps we've learned to it:
- Data Collection
- Data Understanding: Understand the data content and display ``head()``, ``info()``, and ``describe()``.
- Data Integration: Concatenation, Merging and Joining
- Data Statistics: Non numerical Attributes , Numerical Attributes and Categorical Attributes



Thank You

